Reliability Engineering and System Safety 241 (2024) 109682

Contents lists available at ScienceDirect

Reliability Engineering and System Safety

journal homepage: www.elsevier.com/locate/ress

A dynamic mission abort policy for transportation systems with stochastic dependence by deep reinforcement learning

Lujie Liu, Jun Yang *, Bingxin Yan

School of Reliability and Systems Engineering, Beihang University, Beijing, PR China

ARTICLE INFO

Keywords:
Mission abort
Multi-component systems
Stochastic dependence
Functional failure
Deep reinforcement learning
Markov decision process

ABSTRACT

The mission abort policy is considered an effective approach for managing operational risks in transportation systems, which are typically equipped with mission payloads to perform designated tasks. However, existing research mainly focuses on the impact of component failures on system survivability while neglecting situations where the system only experiences functional failures. Additionally, the stochastic dependence between components is always neglected. To address the above problems, a dynamic mission abort method is developed. First, a mission abort policy is proposed to consider the stochastic dependence and mission payload, and the specific mission abort action is determined based on the health state of components and the time in the mission. Next, to maximize the expected cumulative reward during the mission, a dynamic mission abort decision-making model is established based on the Markov decision process. Then, to address the dimensionality curse caused by a continuous state space, a customized deep reinforcement learning method is developed, where the parameter sharing technique is used to reduce the model parameters of the network. Finally, the effectiveness of the proposed method is verified through a numerical example of a reconnaissance UAV, and the superiority of the proposed method is demonstrated by comparing it with heuristic policies.

1. Introduction

Common modes of transportation, such as aircraft, unmanned aerial vehicles (UAVs), and submarines, belong to safety-critical systems, ansd any failures in these systems can result in significant economic losses and even casualties [1]. In practice, transportation systems rely on specific mission payloads to accomplish their tasks, and the failure of these payloads can result in task failure. Examples of such systems include the UAV system for emergency communications [2], the early warning aircraft for reconnaissance, and the submarine for deep-sea exploration [3]. In general, the survival of such systems usually has a higher priority than completing the predetermined task. Therefore, if there is a high risk of task completion, it is better to abort the mission in advance and start the rescue procedure immediately to ensure the survival probability of the system [4]. However, aborting a mission too easily will result in low mission reliability, i.e., the probability of task completion. Therefore, it is essential to determine an appropriate mission abort policy to balance the mission reliability and system survivability.

In 1974, Filene et al. [5] proposed the concept of mission abort to improve crew safety. More recently, Myers et al. [6] studied the mission abort of a warm standby system and regarded the number of failed components as the criteria for mission abort. After that, due to its practical application value, the design and optimization of the mission abort policy are attracting increasing attention from researchers [7,8]. According to the states of the components in the system, the existing researches can be classified into three main categories: binary-state systems [9], multi-state systems [10], and continuous-state systems [11]. For the binary-state system, there are only two possible states for both the system and its components: perfect working and total failure. However, the binary-state assumption is inadequate for capturing the complex characteristics of component or system performance degradation. This assumption oversimplifies the degradation process. By introducing the intermediate states between the perfect working state and the total failure state, the performance degradation process of a component or system can be described more accurately. This allows for a more precise representation of the deterioration that many practical systems experience, as they may operate in various states. As a result, mission abort policies for multi-state systems have also been extensively studied. Furthermore, continuous state degradation systems also exist widely in practical engineering [12], such as microelectromechanical systems

* Corresponding author.
E-mail address: tomyj2001@buaa.edu.cn (J. Yang).

https://doi.org/10.1016/j.ress.2023.109682
Received 2 June 2023; Received in revised form 25 August 2023; Accepted 23 September 2023
Available online 26 September 2023
0951-8320/© 2023 Elsevier Ltd. All rights reserved.

Notation

E_{ij}	the jth component in the ith subsystem
K_i	the number of the components in the ith subsystem is K_i, where $i \in \{1,2\}, j \in \{1,2,\cdots,K_i\}$
K	the total number of components in the system, and $K = K_1 + K_2$
T	the whole mission duration
T_1, T_2	the duration of the task execution phase, and the duration of the return phase
δ	the duration of each inspection interval
$X_n^{E_{ij}}$	the degradation level of component E_{ij} at the time t_n, where $t_n = n\delta$
$\Delta X_n^{E_{ij}}$	the total degradation increment of component E_{ij} from t_n to t_{n+1}
$X_n^{E_{ij},(1)}$	the intrinsic degradation process of component E_{ij} over time
$\Delta X_n^{E_{ij},(1)}$	the intrinsic degradation increment of component E_{ij} from t_n to t_{n+1}
$\Delta X_n^{E_{ij},(2)}$	the degradation increment caused by the degradation levels of the other dependent components from t_n to t_{n+1}
$\boldsymbol{H}$	the degradation interaction matrix
$h_{E_{qw}E_{ij}}$	the impact coefficient of the degradation level of component E_{qw} on the degradation increment of component E_{ij}
$L_{E_{ij}}$	the failure threshold of component $L_{E_{ij}}$
$\varphi(t_n)$	the rescue duration of the system at time t_n
N	the maximum number of decision steps, i.e., $N = \lceil T_1 / \delta \rceil$
Y_n	the operating state of the system at time t_n
$\boldsymbol{S}_n$	the decision state, i.e., $\boldsymbol{S}_n = (\boldsymbol{X}_n^1, \boldsymbol{X}_n^2, n)$
$\boldsymbol{X}_n^1$	the state set of the mission payload
$\boldsymbol{X}_n^2$	the state set of the transportation subsystem
a_n	the action of system at the nth decision epoch, i.e., $a_n \in \{DC, \mathrm{DA}\}$
$P_{sys}(\Psi \mid \boldsymbol{S}_n, \delta)$	the probability of system failure during an inspection interval δ conditional on system state $\boldsymbol{S}_n$
$P_{rs}(\Psi \mid \boldsymbol{S}_n, \varphi(n\delta))$	the probability of system failure during the rescue process conditional on system state $\boldsymbol{S}_n$
$P_{rt}(\Psi \mid \boldsymbol{S}_N, \varphi(T_1))$	the probability of system failure during the return process conditional on system state $\boldsymbol{S}_n$
$P_f(\Psi_{task} \mid \boldsymbol{S}_n, \delta)$	the failure probability of the mission payload during an inspection interval δ conditional on system state $\boldsymbol{S}_n$
$R_{n+1}(\boldsymbol{S}_n, a_n)$	the reward of the $(n+1)$th inspection interval
$V(\boldsymbol{S}_n)$	the value function of the MDP at the state $\boldsymbol{S}_n$
$Q(\boldsymbol{S}_n, a_n)$	the maximum expected cumulative reward of the system within the remaining mission time for the given state $\boldsymbol{S}_n$ and action a_n
$\pi_\theta(\cdot \mid \boldsymbol{s}_n)$	the policy function
c_I	the inspection cost
c_f	the system failure cost
r_m	the profit for completing the predetermined task
c_m	the penalty for not completing the predetermined task
γ	the discount factor
λ	the coefficient for generalized advantage estimation
I	the maximum number of iterations
ξ_i	the ith transition records, i.e., $\xi_i = (\boldsymbol{S}_n, a_n, R_{n+1}, \boldsymbol{S}_{n+1})$
$V_\theta(\boldsymbol{S}_n)$	the approximate value function
τ_n	the temporal difference error
$L_n^{CLIP}(\theta)$	the truncated surrogate objective function for the policy
$L_n^{ET}(\theta)$	the entropy loss function
$L_n^{V}(\theta)$	the loss of the value function
$L_n^{CLIP+ET+V}(\theta)$	the loss function of the unified network
θ	the parameters of the unified network
Z	the number of training epochs
B	the size of minibatch

[13], gearbox systems [14], and tires in a car [15], which are modeled as continuous state systems. With the development of inspection technology, mission abort policies for continuous state systems are gradually attracting more and more attention.

According to the number of components in the system, the objects in the existing research can be divided into single-component systems [16] and multi-component systems [17].

To focus on the components that have a critical impact on system survivability, some research often simplifies the system to a single-component system. Zhao et al. [18] conducted a study on aircraft, considering the cumulative number of valid shocks and the number of consecutive valid shocks, to provide an optimal bivariate mission abort policy for systems operating in random shock environment. Qiu and Cui [19] studied the optimal mission abort policy for UAVs when the component degradation process follows a two-stage Gamma process. Yang et al. [4] investigated the optimal mission abort policy based on the information from early-warning signals and considered the UAV executing the power grid inspection mission as a single-component system. Zhao et al. [20] investigate the joint modeling of loading and mission abort policies for systems operating in a dynamic environment. Qiu et al. [21] studied the optimal condition-based mission abort policy of systems with a two-stage degradation process. Levitin et al. [22] extended the mission abort policy to systems with random rescue time, using a more general non-monotonic threshold function. Levitin et al. [23] advanced the state of the art in aborting policies by modeling systems that perform missions with multiple tasks. Moreover, to minimize the expected total cost, the optimal joint inspection and mission abort policies for continuously operating deteriorating systems were investigated in [24]. One step further, the optimal sampling and mission abort decisions for partially observable safety-critical systems were investigated in [25].

Mission abort policies for multi-component systems have also been extensively studied. Levitin et al. [26] studied the mission abort problem of 1-out-of-N warm standby systems subject to constant or adaptive mission abort policies. Levitin et al. [27] also studied the policy of aborting the mission and performing an emergency landing for a multi-engine aircraft with a certain number of malfunctioned engines to ensure the survival of the aircraft. One step further, the optimal combination of mission abort and standby components arrangement policies for the standby system was studied in [28]. In addition, considering that the expected operational loss depends on the expected number of failed components, the multiple-attempt mission abort policy was extended to the case of multi-component systems [29]. In the case of the limited number of standby components, the joint optimization problem of mission abort and allocation of standby components, considering partial mission loss, was investigated in [3]. Additionally,in the case of the dynamic arrival of tasks with a random amount of work, the joint optimization of mission abort policies and system structure for the warm standby system was investigated in [30]. The balanced system is also a common type of multi-component system, and the related mission abort policy was studied in [31]. Zhao et al. [32] considered the effect of the protection device on system failure and studied the joint optimization policy of mission abort and protective device selection. The two-level rescue depot location policy and dynamic rescue policy were studied in [1], where the UAV was modeled as a multi-component system. The dynamic mission abort policy for a nonrepairable work-sharing system

with a specific workload was studied in [33]. Peng et al. [34] jointly optimized the route and abort policies of multiple cooperative UAVs. One step further, the target, route, and abort policies of UAVs were jointly optimized [9]. Yan et al. [35] studied a joint optimization model that considers task termination when routes of trucks and a drone cluster are assumed under random attacks. Moreover, the dynamic mission abort policy for the swarm was investigated in [36].

Overall, existing research on the mission abort problem of multi-component systems typically assumes that all components within the system are identical [29,37,38]. Furthermore, existing research on the mission abort problem has mainly focused on scenarios where the failure of a component or subsystem has a direct impact on system survivability, while neglecting situations where the system only experiences functional failures.

However, in practical applications, transportation systems are often composed of multiple components with diverse functionalities that work together to achieve the overall system functionality. For example, a UAV system typically consists of a UAV and its mission payload. The UAV is responsible for transporting the mission payload to designated areas. The failure of the mission payload will affect the execution of the task but does not affect the survival of the system. On the other hand, the failure of the UAV will lead to the failure of the entire system. Another example of this issue can be seen in an early warning aircraft that relies on radar to detect distant targets. In such a system, radar failure may lead to task failure without affecting the aircraft's flight. Additionally, the failure of the mechanical arm of a deep-sea submersible may result in the failure of a scientific research mission without affecting its navigation capability. For many modern transportation systems, the mission abort policy that does not consider the impact of mission payload failures may be inadequate. Therefore, it is necessary to study mission abort policies for multi-component systems composed of subsystems with diverse functionalities, as the failure of different subsystems has different impacts on the system.

Furthermore, while the design and optimization of mission abort policies for multi-component systems have been extensively studied, it is commonly assumed that component failures are independent of one another, and their stochastic dependence is often disregarded [39,40]. However, in practice, stochastic dependence exists in many transportation systems [14,41,42]. For example, the gearbox is common in UAVs, aircraft, and submarines. The degradation of bearings in the gearbox may lead to the looseness of the main shaft and increase the vibration level [14]. As a result, the degradation of gears mounted on the main shaft is accelerated due to the impact of vibration. In such cases, a mission abort policy that assumes failure independence between components may be ineffective, and it is necessary to investigate the stochastic dependence between components.

To further advance the state-of-the-art of mission abort policies and extend their practicability, this paper proposes a dynamic mission abort policy for transportation systems that considers the impact of both stochastic dependence between components and the health state of the mission payload. According to the proposed policy, the state transition probabilities of components, subsystems, and the system are derived, taking into account the stochastic dependence between components. Then, to maximize the cumulative reward during the mission, the problem of the mission abort decision is modeled as a Markov decision process (MDP), i.e., the dynamic mission abort decision-making model, which considers the profits from task completion, the costs associated with inspection, system failure, and task failure. Next, to overcome the curse of dimensionality caused by the continuous state space of component states, a customized deep reinforcement learning algorithm, i.e., the DRL-based dynamic mission abort algorithm, is developed to solve the problem. The algorithm utilizes the parameter sharing technique to reduce the model parameters of the network and improve efficiency. Furthermore, the procedures of the proposed algorithm during the training and execution phases are provided. Finally, a comparison is conducted between the proposed method and three heuristic policies to demonstrate its superiority. The main contributions of this work are summarized as follows.

(1) Considering the effects of both stochastic dependence and mission payload on system survival and task completion, a dynamic mission abort policy is proposed.
(2) To maximize the expected cumulative reward, a dynamic mission abort decision-making model based on the MDP is established to implement the proposed policy.
(3) To solve the proposed model with high dimensionality and improve the training efficiency, a customized deep reinforcement learning approach is developed based on the parameter sharing technique.

The rest of this study is organized as follows. Section 2 describes the mission process of a transportation system with stochastic dependence between components and the specific mission payload, as well as the proposed dynamic mission abort policy. The dynamic mission abort decision-making model based on the MDP is established in Section 3. In Section 4, the customized DRL approach based on the parameter sharing technique is developed to optimize the dynamic mission abort policy. Numerical experiments are conducted in Section 5 to verify the effectiveness and superiority of the proposed method. Section 6 summarizes the conclusions and provides directions for future research.

2. Problem description

This section first describes the characteristics of the transportation system with stochastic dependence between components and the specific mission payload. Then, a degradation model with the stochastically dependent component is introduced. Based on the above, a dynamic mission abort policy is proposed.

2.1. System description

Consider a transportation system composed of a transportation subsystem and a specific mission payload. The mission payload is mounted on the transportation subsystem to perform specific tasks, while the transport subsystem is responsible for transporting the mission payload to the target area. For example, in the case of a reconnaissance UAV, the transportation system refers to the entire UAV system, the transportation subsystem refers to the UAV, while the mission payload refers to the device that performs reconnaissance tasks. Let E_{ij} represents the jth component in the ith subsystem and the number of the components in the ith subsystem is K_i, where $i \in \{1,2\}, j \in \{1,2,\cdots,K_i\}$. i equals 1 and 2, which respectively represent the mission payload and the transportation subsystem.

In general, the entire process of a mission can be mainly divided into two phases: the task execution phase and the return phase. In the first phase, the system travels to the target area and performs the designated task. Before completing the task, it is necessary to ensure that both the mission payload and the transportation subsystem are working properly. During the return phase, the system can successfully return to the base, as long as the transportation subsystem does not fail. As shown in Fig. 1, the whole mission duration is denoted as T, and the durations of the two phases are denoted as T_1 and T_2, respectively. It is worth noting that once the system completes the first phase, the predetermined task is accomplished.

According to the types of failure components and their corresponding results, the failures can be classified into the following two categories:

Type 1: The failure of the mission payload will cause task failure due to the inability of the system to perform the specific task. However, it will not affect the survival of the system.

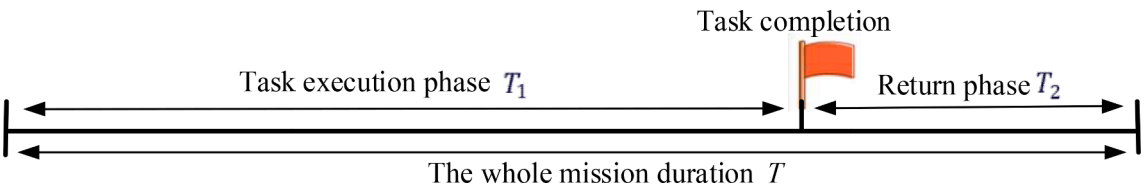


Fig. 1. Description diagram of the mission phase.

Type 2: The failure of the transportation subsystem can directly lead to system failure. Additionally, if the transportation subsystem fails before the task completion, it will also lead to task failure.

Since the failures of the mission payload and transportation subsystem have different impacts on the survival of the system and task completion, it is crucial to consider this difference when determining mission abort policies.

Under the influence of environmental stress, the health condition of components in the system will deteriorate with the increase in operating time. Without loss of generality, we assume that the health condition of the components is inspected periodically every δ units of time by the onboard sensor, and the inspection is perfect.

Furthermore, systems usually consist of multiple components, and their degradation processes are not completely independent of each other. For example, gearboxes commonly used in transportation subsystems typically contain components such as bearings, gears, and shafts. According to the literature [43], the degradation process of the components in the gearbox exhibits significant stochastic dependence. Therefore, it is necessary to also consider the stochastically dependent components in this study.

2.2. Stochastic dependence modeling

Considering that the components in a system have stochastic dependence, i.e., the degradation rate of a component is also affected by the degradation state of all other interdependent components. The degradation process of a component is influenced by both its intrinsic degradation and the degradation state of other components in the system. For instance, the degradation evolution of component E_{ij} can be described as

$$X_{n+1}^{E_{ij}} = X_n^{E_{ij}} + \Delta X_n^{E_{ij}} = X_n^{E_{ij}} + \Delta X_n^{E_{ij},(1)} + \Delta X_n^{E_{ij},(2)}, \tag{1}$$

where $X_n^{E_{ij}}$ and $X_{n+1}^{E_{ij}}$ respectively represent the degradation states of component E_{ij} at the time t_n and t_{n+1}. $\Delta X_n^{E_{ij}}$ is the total degradation increment of component E_{ij} from t_n to t_{n+1}, and it contains two terms: the intrinsic degradation increment of component E_{ij}, i.e., $\Delta X_n^{E_{ij},(1)} = X_n^{E_{ij},(1)} - X_{n-1}^{E_{ij},(1)}$, and the degradation increment caused by the degradation levels of the other interdependent components, i.e., $\Delta X_n^{E_{ij},(2)}$.

According to [44], the degradation increment of the affected component E_{ij} caused by the degradation state of other influencing components can be expressed as

$$\Delta X_n^{E_{ij},(2)} = \sum_{q=1}^{2} \sum_{w \neq j}^{K_q} h_{E_{qw}E_{ij}} \Delta X_n^{E_{qw},(1)}, \tag{2}$$

where $h_{E_{qw}E_{ij}}$ is a specific coefficient to quantify the impact of the degradation level of component E_{qw} on the degradation increment of component E_{ij}, and $\boldsymbol{H}$ represents the degradation interaction matrix, i.e., $\boldsymbol{H} = \{h_{E_{qw}E_{ij}}\}_{K \times K}$, and $K = K_1 + K_2$. In particular, if there is no dependence between the two components in the system, the corresponding impact coefficient is set to 0.

In practice, the intrinsic degradation process of a component is stochastic and can be described by stochastic processes, such as the Gamma process [8], inverse Gaussian (IG) process [45], and Wiener process [46]. Since Wiener processes can be applied to non-monotonic degenerate processes and have good mathematical properties, they are widely used in degradation modeling [45]. Thus, the Wiener process is used to model the degradation of components in this study, and the intrinsic degradation process of component E_{ij} is modeled as a basic linear Wiener process model, i.e.,

$$X_n^{E_{ij},(1)} = X_0^{E_{ij},(1)} + \mu_{E_{ij}} t_n + \sigma_{E_{ij}} B(t_n), \tag{3}$$

where $\mu_{E_{ij}}$ is the drift parameter represents the mean degradation rate, and $\sigma_{E_{ij}}$ is the diffusion coefficient that quantifies the magnitude of the process fluctuation, and $B(t_n)$ is the standard Brownian motion that represents the stochastic degradation process.

According to the additivity of a normal distribution, the overall degradation increment of component E_{ij} during an inspection interval follows a normal distribution, i.e., $\Delta X_n^{E_{ij}} = X_{n+1}^{E_{ij}} - X_n^{E_{ij}} \sim N(\widetilde{\mu}_{E_{ij}}\delta, \widetilde{\sigma}_{E_{ij}}^2\delta)$, and $\widetilde{\mu}_{E_{ij}} = \mu_{E_{ij}} + \sum_{q=1}^{2}\sum_{w\neq j}^{K_q} h_{E_{qw}E_{ij}}\mu_{E_{qw}}$ and $\widetilde{\sigma}_{E_{ij}} = \sqrt{\sigma_{E_{ij}}^2 + \sum_{q=1}^{2}\sum_{w\neq j}^{K_q} h_{E_{qw}E_{ij}}^2 \sigma_{E_{qw}}^2}$. And the probability density function is expressed as

$$f\left(\Delta X_n^{E_{ij}} | \widetilde{\mu}_{E_{ij}}, \widetilde{\sigma}_{E_{ij}}, \delta\right) = \frac{1}{\sqrt{2\pi\widetilde{\sigma}_{E_{ij}}^2\delta}} exp\left(-\frac{\left(\Delta X_n^{E_{ij}} - \widetilde{\mu}_{E_{ij}}\delta\right)^2}{2\widetilde{\sigma}_{E_{ij}}^2\delta}\right). \tag{4}$$

Since the degradation process of the component is modeled as a Wiener process, the failure time of the component follows an IG distribution [47]. The conditional probability of component E_{ij} failing within the $(n+1)$th inspection interval is

$$F_{IG}\left(t_{n+1} - t_n | \frac{L_{E_{ij}} - x}{\widetilde{\mu}_{E_{ij}}}, \frac{\left(L_{E_{ij}} - x\right)^2}{\widetilde{\sigma}_{E_{ij}}^2}\right) = \Phi\left(\frac{\widetilde{\mu}_{E_{ij}}\delta - L_{E_{ij}} + x}{\widetilde{\sigma}_{E_{ij}}\sqrt{\delta}}\right) + exp\left(\frac{2\left(L_{E_{ij}} - x\right)\widetilde{\mu}_{E_{ij}}}{\widetilde{\sigma}_{E_{ij}}^2}\right)\Phi\left(-\frac{\widetilde{\mu}_{E_{ij}}\delta + L_{E_{ij}} - x}{\widetilde{\sigma}_{E_{ij}}\sqrt{\delta}}\right), \tag{5}$$

where t_n and t_{n+1} are the time of the nth and $(n+1)$th inspection, respectively. δ is the duration of each inspection interval, and $L_{E_{ij}}$ is the failure threshold of component E_{ij}.

It is worth noting that for cases where the degradation process of a component follows the Gamma or IG process, the additivity property of the distribution followed by the corresponding degradation increment is only satisfied in specific cases. Additionally, the analytical expressions for the overall degradation increment usually cannot be provided. Therefore, the Monte Carlo simulation method can be used to solve the state transition probability in these cases.

2.3. Mission abort policy

The state of components may deteriorate significantly during the mission. If the system continues to perform the mission, there is a high risk of system failure. Therefore, it is necessary to abort the mission in advance and start the rescue procedure immediately.

Nevertheless, while the mission abort policy can reduce the risk of

system failure, aborting the mission too early also decreases the probability of completing the task. To balance the system survival probability and the probability of task completion, it is necessary to optimize the mission abort policy. Due to differences in the functions of the components in the system, the reasons for mission abort can be divided into two categories:

Type 1: Severe degradation or even failure of the mission payload. The failure of a mission payload can result in the system being unable to perform the specified task, thereby making it unnecessary to continue the mission.
Type 2: Severe degradation of the transportation subsystem. Since the transportation subsystem is responsible for providing power to the entire system and transporting the mission payload to the target area, a failure of the transportation subsystem can cause a system failure. Therefore, it is necessary to abort the mission and start the rescue procedure immediately.

Accordingly, the health state of components in the mission payload and transportation subsystem should be considered comprehensively in the mission abort decision-making. In practice, the duration required for rescue operations can be expressed as a function of the time in the mission, i.e., $\varphi(t_n) = \varphi(n\delta)$. At each inspection, the health state of components and the time in the mission can be utilized to determine whether to continue the current mission.

To better illustrate the effects of the dynamic mission abort policy and the possible scenarios that might occur during the mission, Figs. 2 and 3 show examples of the system operation scenarios based on the proposed mission abort policy. Assume that the system includes a mission payload and a transportation subsystem, where the mission payload consists of one component E_{11} and the transportation subsystem consists of two components E_{21} and E_{22} in series. For this system, if the component in the mission payload fails, the system will be unable to perform the task. Moreover, if any component within the transportation subsystem fails, the entire system will also fail. In Fig. 2, the system successfully completes the task and returns to the base without experiencing any component failures by the end of the mission. In Fig. 3(a), the system aborts the mission due to severe deterioration of the component in the mission payload, and the system is eventually rescued successfully. More specifically, based on the degradation trajectory depicted in the figure, it can be deduced that if the task is not aborted in advance, component E_{11} will fail before reaching time T_1, ultimately leading to the failure of the task. In Fig. 3(b), the system aborts the mission due to the severe deterioration of components in the transportation subsystem, and the system is eventually rescued successfully. More specifically, based on the degradation trajectory of components E_{21} and E_{22} shown in the figure, it can be inferred that if the task is not aborted in advance, the components in the transportation subsystem will fail before the system

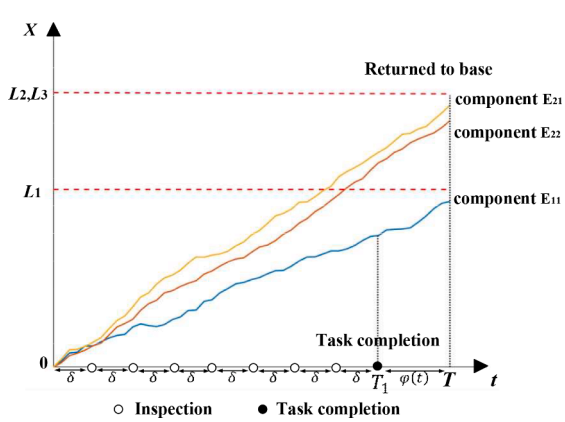


Fig. 2. The system completes the task and returns to the base.

returns to the base. This will ultimately lead to the failure of the entire system. Although proper mission abort policy can improve the survival probability of the system. However, due to the randomness of the component degradation process, system failure may still occur during the rescue process.

3. Dynamic mission abort decision-making model

The dynamic mission abort problem introduced in Section 2.3 can be seen as a sequential decision problem. To find the optimal mission abort policy, the problem is constructed as a finite-horizon MDP in this section. Specifically, the state space, action space, state transition probability, and reward are given. To maximize the expected cumulative reward of the mission, the mission abort action is dynamically determined based on the state of components in the system and the time in the mission. In practice, the decision can only be made at the time of each inspection, which is regarded as a decision epoch in the MDP. It is noteworthy that no decision-making is required during the return process. Therefore, the maximum number of decision epochs is $N = \lceil T_1/\delta \rceil - 1$.

3.1. State space

At the nth decision epoch, the health state of component E_{ij} can be expressed by its continuous degradation state, i.e., $X_n^{E_{ij}}$, $i \in \{1,2\}, j \in \{1, 2, \cdots, K_i\}$. To describe the duration of the system continuously executing the mission, the inspection number n is also integrated into the state space. Since the fixed inspection interval is δ, the corresponding time in the mission is $n\delta$.

According to the description in Section 2.1, Y_n can be divided into four categories: mission execution, mission payload failure, system failure, and successful rescue, which are respectively described with 1 to 4. During the actual operational process, the operating state of the system is determined by the states of its components and the structure of the system. It is important to note that only system in mission execution mode, i.e.., $Y_n = 1$, needs to decide whether to abort the mission. Therefore, there is no need to include the operating state of the system Y_n in the decision state, and the effective state for decision-making is $S_n = (X_n^1, X_n^2, n)$, where $X_n^1 = (X_n^{E_{11}}, X_n^{E_{12}}, \cdots, X_n^{E_{1K_1}})$ is the state set of the mission payload and $X_n^2 = (X_n^{E_{21}}, X_n^{E_{22}}, \cdots, X_n^{E_{2K_2}})$ is the state set of the transportation subsystem.

3.2. Action space

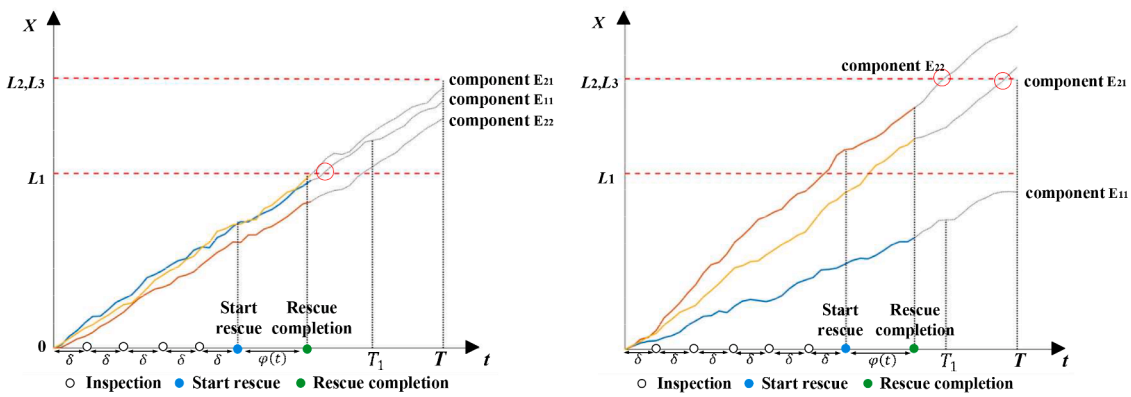
For the system that is executing the mission, there are two optional actions at the nth decision epoch, i.e., $a_n \in \{DA, DC\}$, where DA and DC represent the actions to continue and abort the mission, respectively. Then, depending on the selected action, the system will either continue executing the current mission or immediately start the rescue procedure. In particular, if the mission payload fails, the system will immediately abort the mission.

3.3. State transition probability

Based on the state of the components in the system at the nth inspection and the selected action, the state transitions of the components for the next inspection interval δ can be described as follows. For the components that are executing the mission, there are two possible results. If the component fails before the next inspection, its transition probability is

$$P_n^{E_{ij}}\left(X_{n+1}^{E_{ij}} > L_{E_{ij}} | X_n^{E_{ij}} = x, \delta\right) = F_{IG}\left(\delta | \frac{L_{E_{ij}} - x}{\widetilde{\mu}_{E_{ij}}}, \frac{\left(L_{E_{ij}} - x\right)^2}{\widetilde{\sigma}_{E_{ij}}^2}\right). \tag{6}$$

Otherwise, the transition probability density function of component



(a) Severe degradation of the mission payload (b) Severe degradation of the transportation subsystem

Fig. 3. Two possible scenarios for mission abort.

E_{ij} degenerates to x' less than $L_{E_{ij}}$ is

$$f_n^{E_{ij}}\left(X_{n+1}^{E_{ij}}=x'|X_n^{E_{ij}}=x,\delta\right)=f\left(x'-x|\widetilde{\mu}_{E_{ij}},\widetilde{\sigma}_{E_{ij}},\delta\right). \tag{7}$$

For a system in the return or rescue process, the failure probability of component E_{ij} at the end of this process is

$$\begin{aligned} P_n^{E_{ij}}\left(X_{n+1}^{E_{ij}}>L_{E_{ij}}|X_n^{E_{ij}}=x,\varphi(n\delta)\right) &= F_{IG}\left(\varphi(n\delta)|\frac{L_{E_{ij}}-x}{\widetilde{\mu}_{E_{ij}}},\frac{\left(L_{E_{ij}}-x\right)^2}{\widetilde{\sigma}_{E_{ij}}^2}\right) \\ &= \Phi\left(\frac{\widetilde{\mu}_{E_{ij}}\varphi(n\delta)-L_{E_{ij}}+x}{\widetilde{\sigma}_{E_{ij}}\sqrt{\varphi(n\delta)}}\right)+exp\left(\frac{2\left(L_{E_{ij}}-x\right)\widetilde{\mu}_{E_{ij}}}{\widetilde{\sigma}_{E_{ij}}^2}\right)\Phi\left(-\frac{\widetilde{\mu}_{E_{ij}}\varphi(n\delta)+L_{E_{ij}}-x}{\widetilde{\sigma}_{E_{ij}}\sqrt{\varphi(n\delta)}}\right), \end{aligned} \tag{8}$$

where the duration of the return phase or rescue process is $\varphi(n\delta), n\in\{1,2,\cdots,N\}$. Otherwise, the probability of component E_{ij} operating reliably is

$$\begin{aligned} P_n^{E_{ij}}\left(X_{n+1}^{E_{ij}}\le L_{E_{ij}}|X_n^{E_{ij}}=x,\varphi(n\delta)\right) &= 1-F_{IG}\left(\varphi(n\delta)|\frac{L_{E_{ij}}-x}{\widetilde{\mu}_{E_{ij}}},\frac{\left(L_{E_{ij}}-x\right)^2}{\widetilde{\sigma}_{E_{ij}}^2}\right) \\ &= \Phi\left(\frac{L_{E_{ij}}-\widetilde{\mu}_{E_{ij}}\varphi(n\delta)-x}{\widetilde{\sigma}_{E_{ij}}\sqrt{\varphi(n\delta)}}\right)-exp\left(\frac{2\left(L_{E_{ij}}-x\right)\widetilde{\mu}_q}{\widetilde{\sigma}_{E_{ij}}^2}\right)\Phi\left(-\frac{\widetilde{\mu}_{E_{ij}}\varphi(n\delta)+L_{E_{ij}}-x}{\widetilde{\sigma}_{E_{ij}}\sqrt{\varphi(n\delta)}}\right). \end{aligned} \tag{9}$$

For components with independent degradation processes, the transition probability and probability density function can be obtained directly based on the parameters of their respective stochastic processes, i.e., $\widetilde{\mu}_{E_{ij}}=\mu_{E_{ij}},\widetilde{\sigma}_{E_{ij}}=\sigma_{E_{ij}}$.

In particular, if the system fails or is successfully rescued, it enters the absorbing state and no further state transitions occur.

One step further, based on the state of the components and the structure of the system, the state transition probability of the subsystems and the system during a decision interval δ can be obtained. Without loss of generality, we assume that both the subsystem and the system have series structures, and we take this as an example.

As described in Section 2.3, the failure of the transportation subsystem will result in a system failure, while the failure of the mission payload will only affect its function. For example, the failure of a UAV will result in the crash of both the UAV and its mission payload, leading to the cost of system failure. Therefore, the probability of the system failure during an inspection interval δ depends on the failure probability of the transportation subsystem and is given as

$$P_{sys}(\Psi|\boldsymbol{S}_n,\delta)=\prod_{j=1}^{K_2}P_n^{E_{2j}}\left(X_{n+1}^{E_{2j}}>L_{E_{2j}}|X_n^{E_{2j}}=x,\delta\right). \tag{10}$$

Correspondingly, the probability of system failure during the rescue process is

$$P_{rs}(\Psi|\boldsymbol{S}_n,\varphi(n\delta))=\prod_{j=1}^{K_2}P_n^{E_{2j}}\left(X_{n+1}^{E_{2j}}>L_{E_{2j}}|X_n^{E_{2j}}=x,\varphi(n\delta)\right). \tag{11}$$

Especially, the probability of system failure during the return process is

$$P_{rt}(\Psi|\boldsymbol{S}_N,\varphi(T_1))=\prod_{j=1}^{K_2}P_N^{E_{2j}}\left(X_{N+1}^{E_{2j}}>L_{E_{2j}}|X_N^{E_{2j}}=x,\varphi(T_1)\right). \tag{12}$$

Moreover, the failure probability of the mission payload during an inspection interval is

$$P_f(\Psi_{task}|\boldsymbol{S}_n,\delta)=\prod_{j=1}^{K_1}P_n^{E_{1j}}\left(X_{n+1}^{E_{1j}}>L_{E_{1j}}|X_n^{E_{1j}}=x,\delta\right). \tag{13}$$

It is worth noting that the proposed method is applicable to systems with general structural characteristics, such as series systems, parallel systems, and k-out-of-n systems.

3.4. Reward and Bellman equation

As mentioned in Section 2.3, the system needs to perform periodic inspections during the mission, which incurs the inspection cost c_I. If the system fails during the mission, there will be a failure cost c_f. In addition, completing the predetermined task will generate a certain profit r_m. Otherwise, failing to complete the task will incur a penalty c_m [8]. Specifically, given the state S_n and action a_n at the nth decision epoch, the immediate reward obtained during the next inspection interval can be expressed as

$$R_{n+1}(S_n, a_n) = r_m \cdot I_r(n+1) - c_I \cdot I(a_n = DC) - c_f \cdot I(Y_{n+1} = 3) - c_m \cdot I_c(n+1), \tag{14}$$

where $I_r(n+1)$ indicates that the task is completed during the $(n+1)$th inspection interval, and $I_c(n+1)$ indicates that the task fails during the $(n+1)$th inspection interval. $I(Y_{n+1} = 3)$ indicates that the system failure occurs during the $(n+1)$th inspection interval.

The objective of the MDP is to maximize the expected cumulative reward, which integrates the profit obtained by completing the predetermined task and the costs incurred during the whole mission. The value function of the MDP is denoted as $V(S_n)$, which represents the maximum expected cumulative reward to go from the state S_n. The system has two optional actions in each decision epoch, i.e., $a_n \in \{DA, DC\}$, and $Q(S_n, a_n)$ is denoted as the action value function, i.e., the maximum expected cumulative reward after executing the corresponding action a_n in the state S_n.

For the system specified to abort the task, there are two possible scenarios: (1) The system fails during the return to the base; (2) The system successfully returns to the base. The corresponding action value function can be expressed as $Q(S_n, a_n = DA)$. For the system specified to continue the mission, possible scenarios can be divided into the following four scenarios: (1) The system fails before the next inspection; (2) Only the mission payload fails, but the system is successfully rescued; (3) The mission payload fails and the system is not successfully rescued; (4) Neither the transportation subsystems nor the mission payload fail. The corresponding action value function can be expressed as $Q(S_n, a_n = DC)$. Then, for $n = 1, 2, \cdots, N-1$, the Bellman equation of the MDP can be given as

$$V(S_n) = max\{Q(S_n, a_n = DA), Q(S_n, a_n = DC)\}, \tag{15}$$

where $Q(S_n, a_n = DA)$ is the expected cumulative reward of aborting the mission. If the system fails during the return process, both the costs of system failure and task failure are incurred. Otherwise, only the cost of task failure is incurred. Accordingly, $Q(S_n, a_n = DA)$ is given as

$$Q(S_n, a_n = DA) = -c_m - c_f P_{rs}(\Psi | S_n, \varphi(n\delta)). \tag{16}$$

Moreover, $Q(S_n, a_n = DC)$ is the expected cumulative reward of continuing the mission. The periodic inspection incurs the inspection cost c_I. If the mission payload fails, a task failure cost c_m is incurred. If the system fails, it incurs the costs of system failure and task failure, i.e., c_f and c_m. Otherwise, the system transitions to the next state, i.e., S_{n+1}.

$$\begin{aligned} Q(S_n, a_n = DC) \;=\; & -c_I - (c_f + c_m) P_{sys}(\Psi | S_n, \delta) - c_m (1 - P_{sys}(\Psi | S_n, \delta)) P_f(\Psi_{task} | S_n, \delta)(1 - P_{rs}(\Psi | S_n, \varphi(n\delta))) \\ & - (c_f + c_m)(1 - P_{sys}(\Psi | S_n, \delta)) P_f(\Psi_{task} | S_n, \delta) P_{rs}(\Psi | S_n, \varphi(n\delta)) \\ & + (1 - P_{sys}(\Psi | S_n, \delta))(1 - P_f(\Psi_{task} | S_n, \delta)) \mathscr{M}_n. \end{aligned} \tag{17}$$

where $\mathscr{M}_n = \int_{X_n^{E_{11}}}^{L_{E_{11}}} \int_{X_n^{E_{12}}}^{L_{E_{12}}} \cdots \int_{X_n^{E_{2K_2}}}^{L_{E_{2K_2}}} \prod_{i=1}^{2} \prod_{j=1}^{K_i} f_n^{E_{ij}}(X_{n+1}^{E_{ij}} = x' | X_n^{E_{ij}} = x, \delta) \cdot V(S_{n+1}) dX_{n+1}^{E_{11}} dX_{n+1}^{E_{12}} \cdots dX_{n+1}^{E_{2K_2}}$, and it represents the expected cumulative reward in the subsequent epoch.

Especially, if the system is specified to continue the mission at the Nth decision epoch, i.e., the last decision epoch before task completion, the final result can be divided into five scenarios: 1) The system fails before the task is completed. 2) The mission payload fails before the completion of the task, but the system is successfully rescued. 3) The mission payload fails before the task is completed, and the system fails during the rescue process. 4) The system completes the task but does not successfully return to the base. 5) The system completes the task and successfully returns to the base. Accordingly, $V(S_N)$ can be expressed as follows

$$V(S_N) = max\{Q(S_N, a_N = DA), Q(S_N, a_N = DC)\}. \tag{18}$$

At the Nth decision epoch, the rescue duration is $\varphi(N\delta)$, and the corresponding expected cumulative reward of aborting the mission is

$$Q(S_N, a_N = DA) = -c_m - c_f P_{rs}(\Psi | S_N, \varphi(N\delta)). \tag{19}$$

If the system does not fail during the next inspection interval, the predetermined task is considered completed, and the task completion reward r is obtained, and the duration of return process is $\varphi(T_1)$. The corresponding expected cumulative reward of continuing the mission is

$$\begin{aligned} Q(S_N, a_N = DC) \;=\; & -c_I - (c_f + c_m) P_{sys}(\Psi | S_N, \delta) \\ & - c_m (1 - P_{sys}(\Psi | S_N, \delta)) P_f(\Psi_{task} | S_N, \delta)(1 - P_{rs}(\Psi | S_N, \varphi(N\delta))) \\ & - (c_f + c_m)(1 - P_{sys}(\Psi | S_N, \delta)) P_f(\Psi_{task} | S_N, \delta) P_{rs}(\Psi | S_N, \varphi(N\delta)) \\ & + (r - c_f)(1 - P_{sys}(\Psi | S_N, \delta))(1 - P_f(\Psi_{task} | S_N, \delta)) P_{rt}(\Psi | S_N, \varphi(T_1)) \\ & + r(1 - P_{sys}(\Psi | S_N, \delta))(1 - P_f(\Psi_{task} | S_N, \delta))(1 - P_{rt}(\Psi | S_N, \varphi(T_1))). \end{aligned} \tag{20}$$

Next, the mission abort policy can be obtained by solving the above Bellman equations. Once the mission abort policy is obtained, it can directly output the specific action according to the current state.

4. The customized DRL method for dynamic mission abort

Reinforcement learning (RL) is a branch of machine learning algorithms that can solve MDP problems. The basic idea of RL is that the agent learns a policy to maximize the future cumulative rewards by constantly interacting with the environment through a "trial and error" mechanism. However, tabular RL can only solve small-scale problems. To overcome the limitation of the state space of solved problems, deep learning technology is introduced into RL, and DRL is proposed. Due to

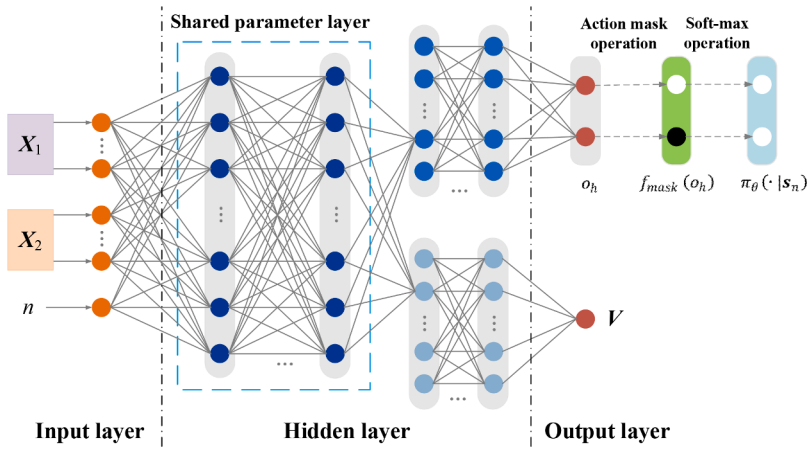


Fig. 4. Illustration of the unified network with multi-head output and parameter sharing.

its ability to solve high-dimensional state and action space problems, DRL has been widely applied in fields such as games, autonomous driving, and operation and maintenance (O&M) management [48,49].

The proposed dynamic mission abort decision-making model, i.e., the MDP model, in Section 3 includes both continuous and discrete states, and the state space is uncountable. Therefore, a customized DRL approach is developed to solve the MDP model and determine the mission abort policy. Specifically, the agent is constructed based on the actor-critic framework in Section 4.1. Section 4.2 presents a customized dynamic mission abort algorithm based on DRL that utilizes the parameter sharing technique.

4.1. The construction of agent in the actor-critic framework

In the typical actor-critic framework, the agent can be divided into two separate parts: the actor, i.e., the policy network, and the critic, i.e., the value network. The actor is responsible for the output of the action at a given state S_n, which determines whether to abort the task. The critic is responsible for evaluating the quality of the current policy. As both the policy network and the value network take the state S_n as input, the parameters of the basic feature extraction network can be shared, i.e., the shallow network can extract general basic features. In this study, a multi-task network is adopted to integrate the actor and critic into a unified network with multi-head output, which can simultaneously output the probabilities of selecting each action and the state value function. By adopting the hard parameter sharing technique [50], the parameters of the network can be reduced. As a result, both the training and inference efficiency of the network can be improved.

We designed a unified neural network model, as shown in Fig. 4. As described in Section 3.1, the effective state for mission abort decision-making is $S_n = (X_n^1, X_n^2, n)$. Therefore, the input layer of the neural network model is the state of components and the time in the mission. As described in Section 3.2, the output layer of the unified network represents the probabilities of selecting each action $a_n \in \{DC, DA\}$ in the given state S_n and the corresponding value function $V(S_n)$. The multi-task network consists of two output heads that output the action and value function in the observed state, with the output layer of each head having 2 and 1 neurons, respectively.

According to the analysis in Section 2.3, when the mission payload fails, the system will immediately abort the current task. To prevent the system from choosing to continue the task when the mission payload fails, a state-dependent action mask layer is added to the neural network of the action output head. The specific implementation of the action mask operation is as follows

$$f_{mask}(o_h) = \begin{cases} -\infty, & \text{if } h = 2 \text{ and the mission payload is failed}, \\ o_h, & \text{otherwise}, \end{cases} \tag{21}$$

where o_h is the original output value of the network. The subscript h is 1 and 2, representing the corresponding output value of DA and DC, respectively. Then, the probability of selecting different actions is finally output by

$$\pi_\theta(a_n = h|s_n) = \frac{exp(f_{mask}(o_h))}{\sum_{h=1}^{2} exp(f_{mask}(o_h))}, \tag{22}$$

where $\pi_\theta(\cdot|s_n)$ is the policy function, which is also known as the softmax distribution. The output generated from the distribution function defined in Eq. (22) is constrained within the numerical range of [0, 1], representing the probabilities of selecting different actions. Moreover, the action with a higher value is more likely to be selected. Especially, according to Eq. (21), if the mission payload fails, the output value corresponding to the action of continuing the task is $-\infty$. Then, the probability of choosing to continue the mission is constrained to 0.

4.2. DRL-based dynamic mission abort algorithm

During the training process of DRL, the agent interacts with the environment to acquire information about the state of components in the system and the time in the mission. Based on this information, the agent selects actions. The environment then provides the next state and the reward obtained from the current decision made by the agent. The goal of the agent is to continuously interact with the environment and learn a policy that maximizes the expected cumulative discounted reward.

During training, the unified network updates the policy and value function by minimizing the loss function, which consists of three parts and is listed as follows

$$L_n^{CLIP+ET+V}(\theta) = \frac{1}{b}\sum_{n=1}^{b}\left[L_n^{CLIP}(\theta) + c_1 L_n^{ET}(\theta) - c_2 L_n^{V}(\theta)\right], \tag{23}$$

where $L_n^{CLIP}(\theta)$ is the truncated surrogate objective function for the policy, $L_n^{ET}(\theta)$ is the entropy loss function, $L_n^{V}(\theta)$ is the loss of the value function. The entropy coefficient c_1 balances exploration and exploitation by encouraging policy exploration through entropy regularization. A higher coefficient promotes more exploration, while a lower coefficient reduces exploration and encourages exploitation. In addition, the value function coefficient c_2 determines the weight given to the value function loss during the overall loss calculation. A higher coefficient prioritizes accurate value estimation. Specifically, the truncated surrogate objective function is given as

$$L_n^{CLIP}(\theta) = min\left(\eta_n(\theta)\widehat{A}_n, f_{clip}(\eta_n(\theta), 1-\varepsilon, 1+\varepsilon)\widehat{A}_n\right), \tag{24}$$

$$\eta_n(\theta) = \frac{\pi_\theta(a_n|\boldsymbol{S}_n)}{\pi_{\theta_{old}}(a_n|\boldsymbol{S}_n)}, \tag{25}$$

where $\eta_n(\theta)$ is the probability ratio between the updated policy and the old policy, which represents the difference between the new policy and the original policy. Specifically, if the updated policy is the same as the old policy, the probability ratio will be equal to 1. The greater the deviation of the probability ratio from 1, the larger the difference between the updated policy and the old policy. To ensure training stability, it is necessary to avoid large differences between the original and new policies [51]. Therefore, a truncation function $f_{clip}(\cdot)$ is used to limit the probability ratio within the range of $[1-\varepsilon, 1+\varepsilon]$, where clip range ε constrains the policy update within a specific range. A larger clip range enables larger policy updates, which can accelerate learning but may introduce instability. Conversely, a smaller clip range ensures more conservative updates, enhancing stability but potentially slowing down learning. Moreover, the advantage function $\widehat{A}_n$ can be calculated using Eq. (26), and the temporal difference error τ_n is given by Eq. (27) [52].

$$\widehat{A}_n = \tau_n + (\gamma\lambda)\tau_{n+1} + \cdots + (\gamma\lambda)^{N-n+1}\tau_{N-1}, \tag{26}$$

$$\tau_n = r_n + \gamma V(\boldsymbol{S}_{n+1}) - V(\boldsymbol{S}_n), \tag{27}$$

where λ is generalized advantage estimation (GAE) coefficient. The GAE coefficient balances bias and variance in advantage estimation. A higher coefficient reduces bias but increases variance, while a lower coefficient reduces variance but introduces bias. In addition, the discount factor γ determines the relative importance of future rewards.

To ensure sufficient exploration of the decision space, the policy entropy $L_n^{ET}(\theta)$ is added to the loss function in Eq. (28). Maximizing policy entropy increases decision randomness and promotes the exploration of different actions under a specific state.

$$L_n^{ET}(\theta) = -\sum_{a_n \in \{DA, DC\}} \pi_\theta(a_n|\boldsymbol{S}_n) log(\pi_\theta(a_n|\boldsymbol{S}_n)). \tag{28}$$

Furthermore, in a given state $\boldsymbol{S}_n = (\boldsymbol{X}_n^1, \boldsymbol{X}_n^2, n)$, the value function can be approximated by $V_\theta(\boldsymbol{S}_n)$, and the performance of the value function approximation can be expressed as

$$L^{VF}(\theta) = \frac{1}{B}\sum_{n=1}^{B}(V_\theta(\boldsymbol{S}_n) - \widehat{V}_n)^2, \tag{29}$$

where $\widehat{V}_n$ is the reward-to-go function that satisfies $\widehat{V}_n = \sum_n^N \gamma^{N-n} R_n$.

The Adam algorithm is chosen to optimize the loss function of the unified network due to its efficiency and adaptivity, as supported by previous research [53]. As the iterations progress, the policy output by the network gradually approaches the optimal mission abort policy, while the approximated value function output gradually approaches the optimal value function.

The pseudocode for training the proposed approach is present in Algorithm 1. At the start of the training, the parameters of the unified network and the algorithm are initialized. The algorithm consists of a total of I iterations, with each iteration having a maximum of N decision epochs. During training, the agent interacts with the environment to sample training data, which is stored in a set $\{B_i\}$ with a limited size of B. At the start of each iteration, the system state is reset to its initial state. More specifically, the current state of the system is obtained through inspection. According to the current state $\boldsymbol{S}_n$, the neural network outputs the values corresponding to different actions, i.e., o_h. Then, using Eqs. (21) and (22), we can obtain the probabilities of selecting different actions, i.e., $\pi_\theta(a_n = h|s_n)$. To facilitate exploration of the agent, during the training process, actions are randomly selected based on their respective probabilities. After executing the selected action a_n, the state of the system transitions from $\boldsymbol{S}_n$ to $\boldsymbol{S}_{n+1}$, and the agent receives an immediate reward R_{n+1}. Subsequently, the trajectory tuple $(\boldsymbol{S}_n, a_n, R_{n+1}, \boldsymbol{S}_{n+1})$ is stored in the replay buffer $\{B_i\}$ for the purpose of training the neural network parameters. Based on the trajectory tuple, the additional information related to the trajectory is calculated, i.e., the reward-to-go, and the advantage estimates. Once each iteration is completed, the network parameters are updated through mini-batch updating with a batch size of b, using the data sampled from the set $\{B_i\}$. Furthermore, the loss function of the neural network can be calculated based on the sampled data using Eqs. (23)-(29). The network parameters are updated for Z epochs per iteration. The selection of the number of epochs should consider the size of the dataset and the computational resources. It is important to note that the optimal values are not fixed and should be adjusted based on empirical observations, prior research, and the specific characteristics of the problem being addressed. Experimentation and tuning are often necessary to determine the most suitable values for these hyperparameters.

As mentioned above, the training process involves the simultaneous learning of both the policy function and the value function through the unified model. This is achieved by minimizing the joint loss function of the unified network. The learning rate determines the step size for updating the policy. It is worth noting that the learning rates of the value network and the policy network are usually inconsistent in the training process. Therefore, a small learning rate should be used in the training process of the unified network with multi-head output.

During the execution phase of the algorithm, the trained network can output actions directly according to the current state $\boldsymbol{S}_n$, and the specific pseudo-code is given in Algorithm 2. It is worth noting that the trained mission abort policy will remain unchanged during the execution phase.

5. Numerical example

In this section, the proposed method is validated using the example of a UAV executing reconnaissance missions in disaster areas. A comparison of policies obtained by considering and ignoring the two factors, i.e., the stochastic dependence between components and the health state of the mission payload, demonstrates the necessity of considering these factors and the effectiveness of the proposed method. Furthermore, the superiority of the proposed method is further demonstrated by comparing it with three heuristic methods.

5.1. Background

UAVs have been widely used in numerous practical engineering scenarios due to their high mobility and flexibility. The UAV system, composed of a mission payload and a transportation subsystem, is extensively applied in practice.

In response to various natural disasters, UAVs are also employed in post-disaster rescue missions [54]. Specifically, reconnaissance UAVs can capture images and video data of the disaster area through their onboard cameras. Additionally, as a critical component of the transportation subsystem, the gearbox exhibits typical performance degradation characteristics, and its failure can directly result in the loss of power of the UAV and subsequent crash. Moreover, the degradation process of the gearbox exhibits significant stochastic dependence [14, 43]. To investigate the impact of the stochastic dependence between components and the health state of mission payloads on mission abort policies, we simplify the UAV system by considering only the camera and gearbox. By considering the health state of the camera and gearbox as inputs, we further analyze the performance of the proposed method.

For convenience, the image sensor in the camera is denoted as E_{11} and the two components in the gearbox are denoted as E_{21}, E_{22}, respectively. Assuming the degradation process of the image sensor is independent and follows the Wiener process, with drift and diffusion parameters of $\mu_{E_{11}} = 0.24$ and $\sigma_{E_{11}} = 0.3$, respectively. The intrinsic degradation behavior of components within a gearbox is also described using a Wiener process, and the corresponding incremental degradation per unit time is assumed to follow a normal distribution with mean $\{\mu_{E_{21}},$

Table 1
Different degradation interaction matrices.

Number	$\boldsymbol{H}0$	$\boldsymbol{H}1$	$\boldsymbol{H}2$	$\boldsymbol{H}3$
Degradation interaction matrix	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \\ 0.008 & 0 & 0.052 \\ 0.008 & 0.043 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \\ 0.01 & 0 & 0.072 \\ 0.01 & 0.063 & 0 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 \\ 0.012 & 0 & 0.092 \\ 0.012 & 0.083 & 0 \end{bmatrix}$

Algorithm 1
DRL-based dynamic mission abort algorithm (Training Phase).

Initialize the parameters of the unified network.
Initialize the replay buffer $\{B_i\}, i = 1, 2, \cdots, I$.
for $i = 1, 2, \cdots, I$ **do**
  Initialize the system state: $\boldsymbol{S} \leftarrow \boldsymbol{S}_0 = (\boldsymbol{X}_0^1, \boldsymbol{X}_0^2, 0), n \leftarrow 0, Y \leftarrow 1, R \leftarrow 0$
  while $Y = 1$ **do**
    The probabilities of different actions being selected are obtained with Eqs. (21) and (22).
    Randomly select an action from the policy $\pi(\cdot|\boldsymbol{S}_n)$
    Execute the selected action
    Obtain state $\boldsymbol{S}_{n+1}, Y = Y_{n+1}$ with Eqs. (6)–(13)
    Compute reward $R_{n+1}(\boldsymbol{S}_n, a_n)$ according to Eq. (14)
    Record the trajectory tuple $(\boldsymbol{S}_n, a_n, R_{n+1}, \boldsymbol{S}_{n+1})$ in set $\{B_i\}$.
    Compute reward-to-go: $\widehat{V}_n = \sum_n^N \gamma^{N-n} R_n$
    Compute advantage estimates $\widehat{A}_n$ with Eq. (26)
    Update the time in the mission
  end while
  for $z = 1, 2, \cdots, Z$ **do**
    Sample a mini-batch b_z of size b from $\{B_i\}$
    Calculate the loss function $L_n^{CLIP+ET+V}(\theta)$ according to Eqs. (23)–(29)
    Update the parameters of the unified network with the mini-batch b_z
  end for
end for

Algorithm 2
DRL-based dynamic mission abort algorithm (Execution Phase).

Obtain the state of the system and the time in the mission
for $n = 1, 2, \cdots, N$ **do**
  Get the state information, i.e., $\boldsymbol{S}_n = (\boldsymbol{X}_n^1, \boldsymbol{X}_n^2, n)$
  Select the action with the highest probability based on the learned policy
  Execute the selected action a_n
  if the system enters the absorption state **then**
    break
  end if
end for

$\mu_{E_{22}}\} = \{0.45, 0.52\}$ and variance $\{\sigma_{E_{21}}, \sigma_{E_{22}}\} = \{0.10, 0.12\}$ parameters. Assuming the degradation state of the image sensor does not affect the degradation process of the components in the gear box. The stochastic dependence between components in the UAV system is represented by a degradation interaction matrix $\boldsymbol{H}2$, i.e., $\boldsymbol{H}2 = \begin{bmatrix} 0 & 0 & 0 \\ 0.01 & 0 & 0.072 \\ 0.01 & 0.063 & 0 \end{bmatrix}$.

The failure thresholds of the components are defined as $\{L_1, L_2, L_3\} = \{7.5, 12, 13\}$. The inspection interval is set as 1. The durations of the two stages are $T_1 = 22$, and $T_2 = 3$, respectively. And the mission duration T is 25. The duration required for the system to return to the base can be expressed as a function of the time in the mission t, i.e., $\varphi(t) = \begin{cases} t, & t \le 3, \\ 3, & 3 \le t \le 22. \end{cases}$

The reward for completing the predetermined task r_m is set as 1200, while a penalty cost of task failure c_m is set as 500. The cost of system failure c_f is set as 1500, and the cost of an inspection during operation, i.e., c_I, is set as 2. Some of the model parameter values are obtained from [8,14,44], and have been slightly adjusted to meet the requirements of the numerical experiments.

5.2. Results and analysis

To assess the effectiveness of the proposed approach, which considers the stochastic dependence between components and the health state of the mission payload, a comparative study is conducted against policies that neglect these factors individually and concurrently. The comparison is performed under the current parameter settings to determine the effectiveness of the proposed method. Furthermore, to analyze the impact of the level of stochastic dependence between components and the degradation rate $\mu_{E_{11}}$ of the mission payload on the effectiveness of the policy, we compared the performance of these policies under different degradation interaction matrices (as shown in Table 1) and degradation rates $[0.20, 0.22, 0.24, 0.26]$.

Fig. 5(a) demonstrates that as the stochastic dependence between components increases, the total rewards of all policies decrease. This occurs because increased dependence leads to more severe degradation of components in the gearbox, which reduces the probability of mission completion. Additionally, as stochastic dependence between components increases, the proposed method shows a greater advantage over other methods, highlighting the importance of considering component stochastic dependence.

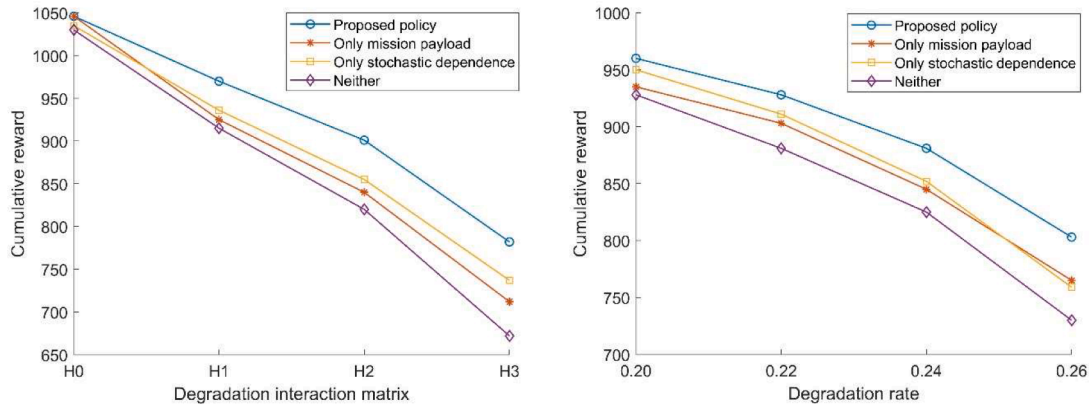
Fig. 5(b) shows that as the degradation rate of the camera sensor increases, all policies exhibit a decrease in the total rewards due to the higher likelihood of mission payload failure, resulting in a lower probability of task completion. Additionally, as the degradation rate increases, the proposed method has a more significant advantage over other methods, particularly for policies that do not consider the health state of the mission payload. This is because the proportion of task failures attributable to mission payload failures is higher. Thus, it is crucial to consider the health state of the mission payload when implementing mission abort policies.

The above results indicate that ignoring the stochastic dependence between components and the health state of the mission payload can affect the effectiveness of mission abort policies. Therefore, it is necessary to consider both the stochastic dependence and the mission payload in the actual implementation of mission abort policies, particularly when the influence of these factors cannot be ignored.

5.3. Policy comparison

To further illustrate the superiority of the proposed dynamic approach, a comparison is made with three heuristic methods. The specific descriptions are listed as follows:

Abort threshold based on fixed degradation ratio (Policy 1): In general, the failure thresholds of each component in a system vary. Therefore, we describe the degradation level of each component by calculating the ratio of its actual degradation level to the corresponding failure threshold, i.e., $ra_q = x_q / L_q$. Under this policy, if the ratio of any component in a system exceeds the predetermined abort threshold ra_0, i.e., $ra_q > ra_0$, the system will abort the current mission and start a rescue



(a) (b)

Fig. 5. Cumulative reward under varying degradation interaction matrices and degradation rates $\mu_{E_{11}}$.

procedure.

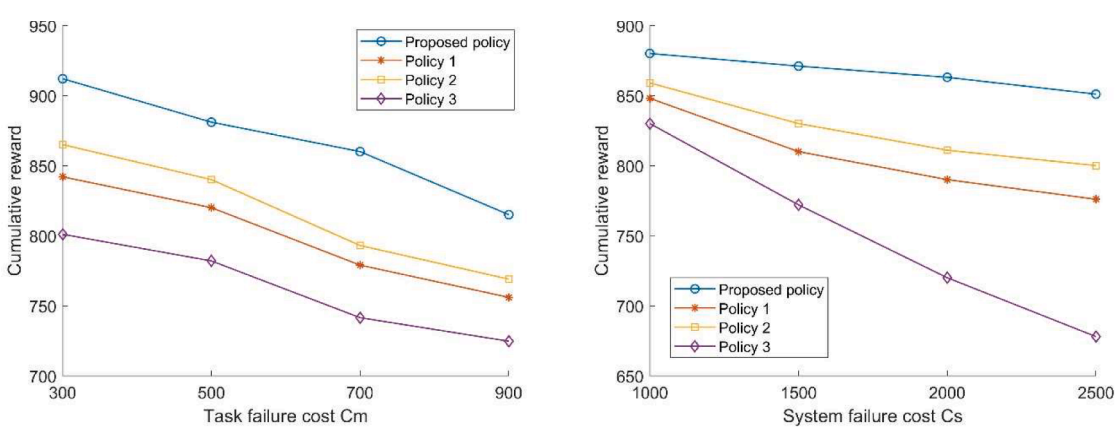
Multiple abort thresholds (Policy 2): Under this policy, the abort thresholds for each component in the system can vary. If the degradation level of any component in the system exceeds its predetermined abort threshold, the system will abort the current mission and start a rescue procedure.

Never abort (Policy 3): Under this policy, mission abort is not performed during the execution process. This is a widely adopted benchmark [7,8].

Next, we will compare the performance of the proposed policy with heuristic policies under different values of c_m and c_f, respectively. Fig. 6 (a) presents the performance of the proposed policy and three heuristic policies as the task failure cost varies. It can be observed that the proposed policy achieves a higher cumulative reward than the other policies at the same task failure cost, demonstrating the superiority of the proposed approach. Furthermore, the cumulative reward of Policy 1 and Policy 2 are both superior to the policy of not considering mission abort, i.e., Policy 3, indicating the necessity of considering mission abort policies. Policy 2 achieves a higher cumulative reward than Policy 1 because it can determine more suitable mission abort criteria based on the degradation characteristics of different components.

Fig. 6 (b) presents how the performance of the proposed policy and three heuristic policies changes with varying system failure costs. The observed superiority of the proposed dynamic mission abort policy over the three heuristic policies under various system failure cost parameter settings demonstrates its effectiveness in reducing system failure losses and improving the cumulative reward. The above results indicate that the proposed dynamic policy that considers the health state of components in the system and the time in the mission is more effective than using a fixed threshold for mission abort policy.

To better illustrate the characteristics of the proposed policy, analyses are conducted on the impact of system failure cost c_f and mission failure cost c_m on the mission abort policy. For scenarios where the rescue duration can be disregarded, we have mathematically proven that when the system failure cost c_f is high, the proposed policy tends to prioritize task completion, i.e., Corollary 1. Conversely, when the mission failure cost c_m is high, the policy tends to focus on managing the operational risks of the system to prevent system failure, i.e., Corollary 2. The specific proofs are presented in Appendix A. Additionally, numerical simulations have been conducted to demonstrate that these findings remain valid when the rescue duration cannot be ignored. More specifically, in Fig. 6 (a), as the task failure cost increases, the decreasing difference in cumulative reward between the proposed policy and Policy 3 suggests that the system is more likely to prioritize completing the current task. In Fig. 6 (b), as the system failure cost increases, the increasing difference in cumulative reward between the proposed policy and Policy 3 suggests that the system tends to prioritize avoiding system failure over completing the task.



(a) The cumulative reward with varying values of c_m (b) The cumulative reward with varying values of c_s

Fig. 6. The performance of policies under varying values of c_m and c_s.

6. Conclusions

This study presents a dynamic mission abort policy that considers the stochastic dependence between components and the health state of the mission payload when making mission abort decisions. To maximize the cumulative reward during the mission, decision-makers control the risk of system failure by determining whether to abort the current mission based on the health condition of components and the time in the mission. Furthermore, to find the optimal mission abort policy, a dynamic mission abort decision-making model is developed based on the MDP. To overcome the dimensionality curse caused by the continuous state space during the solution process, a dynamic mission abort method based on deep reinforcement learning is proposed to solve the optimal policy. In the proposed method, a unified network is utilized to fulfill the roles of both the traditional policy network and the critic network through the parameter sharing technique, thereby reducing model parameters and improving efficiency. The numerical example results demonstrate the importance of considering the stochastic dependence between components and the health state of the mission payload, as well as the effectiveness of the proposed method. Therefore, when the stochastic dependence and degradation effect of the mission payload cannot be ignored, it becomes necessary to consider the influence of these two factors during the implementation of mission abort policies. Additionally, comparisons with other heuristic policies highlight the superiority of the proposed dynamic method. Furthermore, the results indicate that when tasks are important, policies tend to prioritize the task completion. Conversely, when the system value is high, policies tend to focus on managing the operational risks of the system.

In the future, several directions are worth further investigation. First, in practice, the inspection of equipment health condition is often imperfect due to factors such as noise and instrument inaccuracies. Therefore, the current study can be extended to partially observable systems. Secondly, the rescue duration of equipment may be stochastic due to external factors such as environmental conditions. The current study can further consider the scenario where the rescue duration is stochastic. In addition, the mission abort policy considering stochastic dependence under the discrete degradation model is also worth studying.

CRediT authorship contribution statement

Lujie Liu: Conceptualization, Methodology, Writing – original draft, Software, Writing – review & editing. **Jun Yang:** Supervision, Conceptualization, Writing – review & editing, Funding acquisition. **Bingxin Yan:** Conceptualization, Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgement

This work was supported by the National Natural Science Foundation of China under Grant 71971009, JSZL2019204B007, JSZL2017601B006, JSZL2019601B001, and the Fundamental Research Funds for the Central Universities under Grant No. YWF-21-BJ-J-501.

Appendix A

Here is the proof for scenarios where the rescue duration can be neglected. The structural properties of the optimal mission abort policy are investigated.

Definition. Assuming vectors $\boldsymbol{X}$ and $\boldsymbol{W}$ with the same dimension represent two different degradation states of components in a system. If there is at least one element in vector $\boldsymbol{X}$ that is greater than the corresponding element in vector $\boldsymbol{W}$, and all other elements in $\boldsymbol{X}$ are not smaller than the corresponding elements in $\boldsymbol{W}$, we say that vector $\boldsymbol{X}$ is greater than vector $\boldsymbol{W}$.

Theorem 1. *For a fixed n, $V(\boldsymbol{X}, n)$ is a non-increasing function of the degradation states of components $\boldsymbol{X}$. For a fixed $\boldsymbol{X}$, $V(\boldsymbol{X}, n)$ is a non-decreasing function of n.*

Theorem 2. *For a fixed value of n, there exists a threshold X_n^* such that continuing the mission is optimal when $\boldsymbol{X} < X_n^*$, and aborting the mission is optimal when $\boldsymbol{X} > X_n^*$. Furthermore, the abort threshold X_n^* is a non-decreasing function of n.*

Theorem 3. *For a fixed value of $\boldsymbol{X}$, there exists a threshold $n^*(\boldsymbol{X})$ such that continuing the mission is optimal when $n > n^*(\boldsymbol{X})$, and aborting the mission is optimal when $n < n^*(\boldsymbol{X})$. Furthermore, the abort threshold $n^*(\boldsymbol{X})$ is a non-decreasing function of $\boldsymbol{X}$.*

Corollary 1. *The abort threshold $\boldsymbol{X}$ does not increase with the increase of c_f, and does not decrease with the increase of c_m.*

Corollary 1 suggests that, when the system value is high, the mission should be aborted at a relatively lower degradation state to mitigate operational risks. Conversely, when the task is important, a relatively higher degradation-based abort threshold is preferred to prioritize task completion.

Corollary 2. *The abort threshold $n^*(\boldsymbol{X})$ does not decrease with the increase of c_f, and does not increase with the increase of c_m.*

Corollary 2 states that when the system failure cost increases, the mission should be aborted earlier to reduce the probability of system failure. Conversely, if the cost of mission failure is higher, the mission can be extended for a longer time to improve the probability of mission completion.

Proof of Theorem 1

Denote $\langle X_{n+1}^{E_{11}}, X_{n+1}^{E_{12}}, \cdots, X_{n+1}^{E_{2K_2}} | X_n^{E_{11}} = x^{E_{11}}, X_n^{E_{12}} = x^{E_{12}}, \cdots, X_n^{E_{2K_2}} = x^{E_{2K_2}} \rangle$ as the state of the components in the system at the $(n+1)$th inspection. First, consider the degradation state of the component E_{11}. For $x^{E_{11}} < x^{E_{11}} +$, it follows that

$$\begin{aligned}
&Pr\left\{X_{n+1}^{E_{11}} \le L_{E_{11}}, X_{n+1}^{E_{12}} \le L_{E_{12}}, \cdots, X_{n+1}^{E_{2K2}} \le L_{E_{2K2}} | X_n^{E_{11}} = x^{E_{11}} +, X_n^{E_{12}} = x^{E_{12}}, \cdots, X_n^{E_{2K2}} = x^{E_{2K2}}\right\} \\
&= Pr\left\{X_{n+1}^{E_{11}} - X_n^{E_{11}} \le L_{E_{11}} - x^{E_{11}} +, X_{n+1}^{E_{12}} - X_n^{E_{12}} \le L_{E_{12}} - x^{E_{12}}, \cdots, X_{n+1}^{E_{2K2}} - X_n^{E_{2K2}} \le L_{E_{2K2}} - x^{E_{2K2}}\right\} \\
&\le Pr\left\{X_{n+1}^{E_{11}} - X_n^{E_{11}} \le L_{E_{11}} - x^{E_{11}}, X_{n+1}^{E_{12}} - X_n^{E_{12}} \le L_{E_{12}} - x^{E_{12}}, \cdots, X_{n+1}^{E_{2K2}} - X_n^{E_{2K2}} \le L_{E_{2K2}} - x^{E_{2K2}}\right\} \\
&= Pr\left\{X_{n+1}^{E_{11}} \le L_{E_{11}}, X_{n+1}^{E_{12}} \le L_{E_{12}}, \cdots, X_{n+1}^{E_{2K2}} \le L_{E_{2K2}} | X_n^{E_{11}} = x^{E_{11}}, X_n^{E_{12}} = x^{E_{12}}, \cdots, X_n^{E_{2K2}} = x^{E_{2K2}}\right\}
\end{aligned} \tag{A.1}$$

If the initial degradation state of component E_{11} is higher, the probability of it not failing in the next inspection decreases. Additionally, as the deterioration state of component E_{11} increases, the degradation increment experienced by other components within the unit inspection interval will also be larger. Therefore, the probability of components in the system not failing in the next inspection decreases.

According to the definition of stochastic order, we have $\langle X_{n+1}^{E_{11}}, X_{n+1}^{E_{12}}, \cdots, X_{n+1}^{E_{2K2}} | X_n^{E_{11}} = x^{E_{11}} +, X_n^{E_{12}} = x^{E_{12}}, \cdots, X_n^{E_{2K}} 2 = x^{E_{2K2}} \rangle \succ_{st} \langle X_{n+1}^{E_{11}}, X_{n+1}^{E_{12}}, \cdots, X_{n+1}^{E_{2K2}} | X_n^{E_{11}} = x^{E_{11}}, X_n^{E_{12}} = x^{E_{12}}, \cdots, X_n^{E_{2K2}} = x^{E_{2K2}} \rangle$. Similarly, we can also prove $\langle X_{n+1}^{E_{11}}, X_{n+1}^{E_{12}}, \cdots, X_{n+1}^{E_{2K2}} | X_n^{E_{11}} = x^{E_{11}^+}, X_n^{E_{12}} = x^{E_{12}^+}, \cdots, X_n^{E_{2K2}} = x^{E_{2K2}^+} \rangle \succ_{st} \langle X_{n+1}^{E_{11}}, X_{n+1}^{E_{12}}, \cdots, X_{n+1}^{E_{2K2}} | X_n^{E_{11}} = x^{E_{11}}, X_n^{E_{12}} = x^{E_{12}}, \cdots, X_n^{E_{2K2}} = x^{E_{2K2}} \rangle$.

To facilitate subsequent proofs, we define the condition where at least one component in the system has a degradation state greater than $\boldsymbol{X}$ as $\boldsymbol{X}^+$, where $\boldsymbol{X} = (x^{E_{11}}, x^{E_{12}}, \cdots, x^{E_{2K_2}})$.

Next, the monotonicity property of the value function is proved by mathematical induction.

First, we prove that the value function does not increase with the degradation state of the component. For the Nth decision epoch, the value function is constant, which is obviously nonincreasing with respect to the degradation state of the component. Assuming that this property holds at the $(n+1)$th decision epoch and $n \le N-1$, we should prove that this property also holds at the nth decision epoch. According to the definition of $Q(\boldsymbol{X}, n, a_n = DA)$, it is evident that it is nonincreasing with respect to the degradation state of the component. Furthermore, we can prove that $Q(\boldsymbol{X}, n, a_n = DC)$ is nonincreasing with respect to the component degradation state. More specifically, as shown below.

Assume $V(\boldsymbol{X}, n+1)$ is nonincreasing with respect to $\boldsymbol{X}$, $\mathscr{M}_n = \int_{X_n^{E_{11}}}^{L_{E_{11}}} \int_{X_n^{E_{12}}}^{L_{E_{12}}} \cdots \int_{X_n^{E_{2K_2}}}^{L_{E_{2K_2}}} \prod_{i=1}^{2} \prod_{j=1}^{K_i} f_n^{E_{ij}} (X_{n+1}^{E_{ij}} = x' | X_n^{E_{ij}} = x, \delta) \cdot V(\boldsymbol{X}, n+1) dX_{n+1}^{E_{11}} dX_{n+1}^{E_{12}} \cdots dX_{n+1}^{E_{2K_2}}$ is also nonincreasing with respect to $\boldsymbol{X}$. Hence, it is easy to prove that the value of $Q(\boldsymbol{X}, n, a_n = DC)$ is nonincreasing with respect to $\boldsymbol{X}$ according to the definitions of the terms in $Q(\boldsymbol{X}, n, a_n = DC)$. One step further, we can have $V(\boldsymbol{X}, n)$ is non-increasing with respect to $\boldsymbol{X}$. From this, it can be concluded that the value function $V(\boldsymbol{X}, n)$ is non-increasing with respect to $\boldsymbol{X}$.

Second, we prove that the value function is nondecreasing with respect to n. Assuming $V(\boldsymbol{X}, n) \le V(\boldsymbol{X}, n+1)$, for a fixed component degradation state, $\mathscr{M}_{n-1} \le \mathscr{M}_n$. Since the other terms in $Q(\boldsymbol{X}, n, a_n = DC)$ are independent of n, $Q(\boldsymbol{X}, n-1, a_{n-1} = DC) \le Q(\boldsymbol{X}, n, a_n = DC)$. One step further, we can have $V(\boldsymbol{X}, n-1) = max\{Q(\boldsymbol{X}, n-1, a_{n-1} = DA), Q(\boldsymbol{X}, n-1, a_{n-1} = DC)\} \le max\{Q(\boldsymbol{X}, n, a_n = DA), Q(\boldsymbol{X}, n, a_n = DC)\} = V(\boldsymbol{X}, n)$. From this, it can be concluded that the value function $V(\boldsymbol{X}, n)$ is nondecreasing with respect to n.

Proof of Theorem 2

To establish an optimal control limit for a given n, the approach based on dynamic programming involves proving that $Q(\boldsymbol{X}, n, a_n = DC) - Q(\boldsymbol{X}, n, a_n = DA)$ is decreasing with respect to $\boldsymbol{X}$. This implies that if this difference exceeds 0 at a certain $\boldsymbol{X}$, it becomes optimal to abort the mission for all larger values of n. Therefore, the sufficient condition for the existence of the control limit in $\boldsymbol{X}$ for a fixed n is

$$Q(\boldsymbol{X}, n, a_n = DC) - Q(\boldsymbol{X}, n, a_n = DA) \ge Q(\boldsymbol{X}^+, n, a_n = DC) - Q(\boldsymbol{X}^+, n, a_n = DA) \tag{A.2}$$

Since $Q(\boldsymbol{X}, n, a_n = DC)$ is a non-increasing function of $\boldsymbol{X}$ according to Theorem 1, then we have

$$Q(\boldsymbol{X}, n, a_n = DC) - Q(\boldsymbol{X}^+, n, a_n = DC) \ge Q(\boldsymbol{X}, n, a_n = DA) - Q(\boldsymbol{X}^+, n, a_n = DA) = 0 \tag{A.3}$$

Hence, the sufficient condition holds. For a fixed n, there exists a threshold $\boldsymbol{X}^*(n)$ such that for $\boldsymbol{X} < \boldsymbol{X}^*(n)$, it is optimal to continue the mission, and for $\boldsymbol{X} > \boldsymbol{X}^*(n)$, it is optimal to abort the mission.

Then we prove the monotonicity of $\boldsymbol{X}^*(n)$. If $Q(\boldsymbol{X}, n, a_n = DC) \ge Q(\boldsymbol{X}, n, a_n = DA)$, then for all $n \le g$, we have $Q(\boldsymbol{X}, g, a_g = DC) \ge Q(\boldsymbol{X}, n, a_n = DA)$ since $Q(\boldsymbol{X}, n, a_n = DC)$ is non-decreasing in n. Thus, $\boldsymbol{X}^*(n) \le \boldsymbol{X}^*(g)$, and the abort threshold $\boldsymbol{X}^*(n)$ is a non-decreasing function of n.

Proof of Theorem 3

The sufficient condition for the existence of an optimal control limit in n for a fixed $\boldsymbol{X}$ is

$$Q(\boldsymbol{X}, n+1, a_{n+1} = DC) - Q(\boldsymbol{X}, n+1, a_{n+1} = DA) \ge Q(\boldsymbol{X}, n, a_n = DC) - Q(\boldsymbol{X}, n, a_n = DA) \tag{A.4}$$

Since $Q(\boldsymbol{X}, n, a_n = DC)$ is a non-decreasing function of n according to Theorem 1, then we have

$$Q(\boldsymbol{X}, n+1, a_n = DC) - Q(\boldsymbol{X}, n, a_n = DC) \ge Q(\boldsymbol{X}, n+1, a_n = DA) - Q(\boldsymbol{X}, n, a_n = DA) = 0 \tag{A.5}$$

Hence, the sufficient condition holds. For a fixed $\boldsymbol{X}$, there exists $n^*(\boldsymbol{X})$ such that for $n < n^*(\boldsymbol{X})$, it is optimal to abort the mission, and for $n > n^*(\boldsymbol{X})$, it is optimal to continue the mission.

If $Q(\boldsymbol{X}, n, a_n = DC) \le Q(\boldsymbol{X}, n, a_n = DA)$, then for all $\boldsymbol{X} \le \boldsymbol{X}^+$, we have $Q(\boldsymbol{X}^+, n, a_n = DC) \le Q(\boldsymbol{X}^+, n, a_n = DA)$ because $Q(\boldsymbol{X}, n, a_n = DC)$ is non-increasing in $\boldsymbol{X}$. Therefore, $n^*(\boldsymbol{X}) \le n^*(\boldsymbol{X}^+)$ for all $\boldsymbol{X} \le \boldsymbol{X}^+$, and the abort threshold $n^*(\boldsymbol{X})$ is a non-decreasing function of $\boldsymbol{X}$.

Proof of Corollary 1

Denote two system failure costs c_{f_1} and c_{f_2}, where $c_{f_1} < c_{f_2}$. Let $\boldsymbol{X}_1^*(n)$ and $\boldsymbol{X}_2^*(n)$ be the optimal abort thresholds under c_{f_1} and c_{f_2}, respectively. Suppose $\boldsymbol{X}_1^*(n) < \boldsymbol{X}_2^*(n)$. According to Theorem 1, for $\boldsymbol{X} < \boldsymbol{X}_2^*(n)$ it is optimal to continue the mission under system failure cost c_{f_2}, and we have

$$\begin{aligned}
Q\left(\boldsymbol{X}_1^*(n), n, a_n = DC | c_{f_2}\right) \quad = \quad & -c_I - \left(c_{f_2} + c_m\right) P_{sys}\left(\Psi | \boldsymbol{X}_1^*(n), n, \delta\right) \\
& - c_m\left(1 - P_{sys}\left(\Psi | \boldsymbol{X}_1^*(n), n, \delta\right)\right) P_f\left(\Psi_{task} | \boldsymbol{X}_1^*(n), n, \delta\right) - \left(c_{f_2} + c_m\right)\left(1 - P_{sys}\left(\Psi | \boldsymbol{X}_1^*(n), n, \delta\right)\right) P_f\left(\Psi_{task} | \boldsymbol{X}_1^*(n), n, \delta\right) \\
& + \left(1 - P_{sys}\left(\Psi | \boldsymbol{X}_1^*(n), n, \delta\right)\right)\left(1 - P_f\left(\Psi_{task} | \boldsymbol{X}_1^*(n), n, \delta\right)\right) \mathscr{M}_n > Q\left(\boldsymbol{X}_2^*(n), n, a_n = DA | c_{f_2}\right).
\end{aligned} \tag{A.6}$$

Since $c_{f_1} < c_{f_2}$, then the following inequality holds

$$\begin{aligned}
&Q\left(X_1^*(n),n,a_n=DC|c_{f2}\right)=-c_I-\left(c_{f2}+c_m\right)P_{sys}\left(\Psi|X_1^*(n),n,\delta\right)-c_m\left(1-P_{sys}\left(\Psi|X_1^*(n),n,\delta\right)\right)\\
&P_f\left(\Psi_{task}|X_1^*(n),n,\delta\right)-\left(c_{f2}+c_m\right)\left(1-P_{sys}\left(\Psi|X_1^*(n),n,\delta\right)\right)P_f\left(\Psi_{task}|X_1^*(n),n,\delta\right)+\left(1-P_{sys}\left(\Psi|X_1^*(n),n,\delta\right)\right)\left(1-P_f\left(\Psi_{task}|X_1^*(n),n,\delta\right)\right)\\
&\mathscr{M}_n\le -c_I-\left(c_{f1}+c_m\right)P_{sys}\left(\Psi|X_1^*(n),n,\delta\right)-c_m\left(1-P_{sys}\left(\Psi|X_1^*(n),n,\delta\right)\right)P_f\left(\Psi_{task}|X_1^*(n),n,\delta\right)-\left(c_{f1}+c_m\right)\left(1-P_{sys}\left(\Psi|X_1^*(n),n,\delta\right)\right)\\
&P_f\left(\Psi_{task}|X_1^*(n),n,\delta\right)+\left(1-P_{sys}\left(\Psi|X_1^*(n),n,\delta\right)\right)\left(1-P_f\left(\Psi_{task}|X_1^*(n),n,\delta\right)\right).\mathscr{M}_n=Q\left(X_1^*(n),n,a_n=DC|c_{f1}\right)\left\langle Q\left(X_1^*(n),n,a_n=DA|c_{f1}\right)\right.
\end{aligned}\tag{A.7}$$

The above two inequalities are equivalent to $Q(X_1^*(n),n,a_n=DC|c_{f_2})>-c_m$ and $Q(X_1^*(n),n,a_n=DC|c_{f_2})<-c_m$, which is a contradiction. Thus, $X_1^*(n)\ge X_2^*(n)$, i.e., $X^*(n)$ does not increase as c_f increases. Similarly, $X^*(n)$ can also be proved to be nondecreasing in c_m.

Proof of Corollary 2

Denote two system failure costs c_{f_1} and c_{f_2}, where $c_{f_1}<c_{f_2}$. Let $n_1^*(X)$ and $n_2^*(X)$ be the optimal abort thresholds under c_{f_1} and c_{f_2}, respectively. Suppose $n_1^*(X)>n_2^*(X)$. According to Theorem 1, for $n>n_2^*(X)$ it is optimal to continue the mission under the system failure cost c_{f_2}, and we have

$$\begin{aligned}
Q\left(X,n_1^*(X),a_n=DC|c_{f_2}\right)=&-c_I-\left(c_{f_2}+c_m\right)P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)-c_m\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)\\
&-\left(c_{f_2}+c_m\right)\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)+\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)\left(1-P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)\right).\mathscr{M}_n\\
&>Q\left(X,n_2^*(X),a_n=DA|c_{f_2}\right).
\end{aligned}\tag{A.8}$$

Since $c_{f_1}<c_{f_2}$, then the following inequality holds

$$\begin{aligned}
&Q\left(X,n_1^*(X),a_n=DC|c_{f2}\right)=-c_I-\left(c_{f2}+c_m\right)P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)-c_m\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)-\left(c_{f2}+c_m\right)\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)\\
&\times P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)+\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)\\
&\left(1-P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)\right).\mathscr{M}_n\le -c_I-\left(c_{f2}+c_m\right)P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)-c_m\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)-\left(c_{f1}+c_m\right)\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)\\
&P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)+\left(1-P_{sys}\left(\Psi|X,n_1^*(X),\delta\right)\right)\left(1-P_f\left(\Psi_{task}|X,n_1^*(X),\delta\right)\right).\mathscr{M}_n=Q\left(X,n_1^*(X),a_n=DC|c_{f1}\right)\left\langle Q\left(X,n_1^*(X),a_n=DA|c_{f1}\right)\right.
\end{aligned}\tag{A.9}$$

The above two inequalities are equivalent to $Q(X,n_1^*(X),a_n=DC|c_{f_2})>-c_m$ and $Q(X,n_1^*(X),a_n=DC|c_{f_2})<-c_m$, which is a contradiction. Thus, $n_1^*(X)\le n_2^*(X)$, i.e., $n^*(X)$ does not decrease as c_f increases. Similarly, $n^*(X)$ can also be proved to be nonincreasing in c_m.

References

[1] Zhao X, Lv Z, Qiu Q, Wu Y. Designing two-level rescue depot location and dynamic rescue policies for unmanned vehicles. Reliab Eng Syst Saf 2023;233:109119.
[2] Mayor V, Estepa R, Estepa A, Madinabeitia G. Deploying a reliable UAV-aided communication service in disaster areas. Wirel Commun Mob Comput 2019;2019: 1–20.
[3] Zhao X, Dai Y, Qiu Q, Wu Y. Joint optimization of mission aborts and allocation of standby components considering mission loss. Reliab Eng Syst Saf 2022;225: 108612.
[4] Yang L, Sun Q, Ye Z. Designing mission abort strategies based on early-warning information: application to UAV. IEEE Trans Ind Inform 2020;16:277–87.
[5] Filene R, Daly W. The Reliability impact of mission abort strategies on redundant flight computer systems. IEEE Trans Comput 1974;C–23:739–43.
[6] Myers A. Probability of loss assessment of critical k-out-of-n :G systems having a mission abort policy. IEEE Trans Reliab 2009;58:694–701.
[7] Yang L, Chen Y, Qiu Q, Wang J. Risk control of mission-critical systems: abort decision-makings integrating health and age conditions. IEEE Trans Ind Inform 2022;18:6887–94.
[8] Zhao X, Sun J, Qiu Q, Chen K. Optimal inspection and mission abort policies for systems subject to degradation. Eur J Oper Res 2021;292:610–21.
[9] Zhu X, Yan R, Peng R, Zhang Z. Optimal routing, loading and aborting of UAVs executing both visiting tasks and transportation tasks. Reliab Eng Syst Saf 2020; 204:107132.
[10] Levitin G, Finkelstein M, Xiang Y. Optimal mission abort policies for repairable multistate systems performing multi-attempt mission. Reliab Eng Syst Saf 2021; 209:107497.
[11] Zhao X, Fan Y, Qiu Q, Chen K. Multi-criteria mission abort policy for systems subject to two-stage degradation process. Eur J Oper Res 2021;295:233–45.
[12] Wu B, Wei X, Zhang Y, Bai S. Modeling dynamic environment effects on dependent failure processes with varying failure thresholds. Reliab Eng Syst Saf 2023;229: 108848.
[13] Wang J, Li Z, Bai G, Zuo M. An improved model for dependent competing risks considering continuous degradation and random shocks. Reliab Eng Syst Saf 2020; 193:106641.
[14] Bian L, Gebraeel N. Stochastic modeling and real-time prognostics for multi-component systems with degradation rate interactions. IIE Trans 2014;46:470–82.
[15] Sun Q, Ye Z, Zhu X. Managing component degradation in series systems for balancing degradation through reallocation and maintenance. IISE Trans 2020;52: 797–810.
[16] Qiu Q, Cui L, Wu B. Dynamic mission abort policy for systems operating in a controllable environment with self-healing mechanism. Reliab Eng Syst Saf 2020; 203:107069.
[17] Levitin G, Finkelstein M, Xiang Y. Optimal aborting strategy for three-phase missions performed by multiple units. Reliab Eng Syst Saf 2021;208:107408.
[18] Zhao X, Chai X, Sun J, Qiu Q. Optimal bivariate mission abort policy for systems operate in random shock environment. Reliab Eng Syst Saf 2021;205:107244.
[19] Qiu Q, Cui L. Gamma process based optimal mission abort policy. Reliab Eng Syst Saf 2019;190:106496.
[20] Zhao X, Li R, Cao S, Qiu Q. Joint modeling of loading and mission abort policies for systems operating in dynamic environments. Reliab Eng Syst Saf 2023;230: 108948.
[21] Qiu Q, Maillart L, Prokopyev O, Cui L. Optimal condition-based mission abort decisions. IEEE Trans Reliab 2023;72:408–25.
[22] Levitin G, Xing L, Dai Y. Optimal aborting policy for shock exposed missions with random rescue time. Reliab Eng Syst Saf 2023;233:109094.
[23] Levitin G, Xing L, Dai Y. Optimal task sequencing and aborting in multi-attempt multi-task missions with a limited number of attempts. Reliab Eng Syst Saf 2023; 236:109309.
[24] Cheng G, Li L, Zhang L, Yang N, Jiang B, Shangguan C, et al. Optimal joint inspection and mission abort policies for degenerative systems. IEEE Trans Reliab 2023;72:137–50.
[25] Yang L, Wei F, Qiu Q. Mission risk control via joint optimization of sampling and abort decisions. Risk Anal 2023. risa.14187.
[26] Levitin G, Finkelstein M. Optimal mission abort policy for systems operating in a random environment: optimal mission abort policy. Risk Anal 2018;38:795–803.
[27] Levitin G, Xing L, Dai Y. Mission abort policy in heterogeneous nonrepairable 1-out-of-N warm standby systems. IEEE Trans Reliab 2018;67:342–54.
[28] Levitin G, Finkelstein M, Li Y. Balancing mission success probability and risk of system loss by allocating redundancy in systems operating with a rescue option. Reliab Eng Syst Saf 2020;195:106694.
[29] Levitin G, Finkelstein M, Xiang Y. Optimal aborting rule in multi-attempt missions performed by multicomponent systems. Eur J Oper Res 2020;283:244–52.
[30] Zhao X, Liu H, Wu Y, Qiu Q. Joint optimization of mission abort and system structure considering dynamic tasks. Reliab Eng Syst Saf 2023;234:109128.
[31] Wang S, Zhao X, Tian Z, Zuo MJ. Optimal mission abort policy with multiple abort criteria for a balanced system with multi-state components. Comput Ind Eng 2021; 160:107544.
[32] Zhao X, Chai X, Sun J, Qiu Q. Joint optimization of mission abort and protective device selection policies for multistate systems. Risk Anal 2022;42:2823–34.
[33] Levitin G, Xing L, Dai Y. Mission aborting in n -unit systems with work sharing. IEEE Trans Syst Man Cybern Syst 2022;52:4875–86.
[34] Peng R. Joint routing and aborting optimization of cooperative unmanned aerial vehicles. Reliab Eng Syst Saf 2018;177:131–7.
[35] Yan R, Zhu X, Zhu X, Peng R. Joint optimisation of task abortions and routes of truck-and-drone systems under random attacks. Reliab Eng Syst Saf 2023;235: 109249.
[36] Liu L, Yang J. A dynamic mission abort policy for the swarm executing missions and its solution method by tailored deep reinforcement learning. Reliab Eng Syst Saf 2023;234:109149.
[37] Wu C, Zhao X, Qiu Q, Sun J. Optimal mission abort policy for k-out-of-n: F balanced systems. Reliab Eng Syst Saf 2021;208:107398.
[38] Zhao X, Chai X, Sun J, Qiu Q. Joint optimization of mission abort and component switching policies for multistate warm standby systems. Reliab Eng Syst Saf 2021; 212:107641.
[39] Levitin G, Finkelstein M, Dai Y. State-based mission abort policies for multistate systems. Reliab Eng Syst Saf 2020;204:107122.
[40] Qiu Q, Cui L. Optimal mission abort policy for systems subject to random shocks based on virtual age process. Reliab Eng Syst Saf 2019;189:11–20.

[41] Liu B, Pandey M, Wang X, Zhao X. A finite-horizon condition-based maintenance policy for a two-unit system with dependent degradation processes. Eur J Oper Res 2021;295:705–17.
[42] Kong X, Yang J, Li L. Reliability analysis for multi-component systems considering stochastic dependency based on factor analysis. Mech Syst Signal Process 2022; 169:108754.
[43] Liang X, Zuo M, Feng Z. Dynamic modeling of gearbox faults: a review. Mech Syst Signal Process 2018;98:852–76.
[44] Wang Y, Li X, Chen J, Liu Y. A condition-based maintenance policy for multi-component systems subject to stochastic and economic dependencies. Reliab Eng Syst Saf 2022;219:108174.
[45] Ye Z, Xie M. Stochastic modelling and analysis of degradation for highly reliable products. Appl Stoch Models Bus Ind 2015;31:16–32.
[46] Hu J, Shen J, Shen L. Opportunistic maintenance for two-component series systems subject to dependent degradation and shock. Reliab Eng Syst Saf 2020;201: 106995.
[47] Wang H, Ma X, Zhao Y. An improved Wiener process model with adaptive drift and diffusion for online remaining useful life prediction. Mech Syst Signal Process 2019;127:370–87.
[48] Zhang P, Zhu X, Xie M. A model-based reinforcement learning approach for maintenance optimization of degrading systems in a large state space. Comput Ind Eng 2021;161:107622.
[49] Liu Y, Gao J, Jiang T, Zeng Z. Selective maintenance and inspection optimization for partially observable systems: an interactively sequential decision framework. IISE Trans 2023;55:463–79.
[50] Ruder S. An overview of multi-task learning in deep neural networks. arXiv preprint 2017. arXiv:1706.05098.
[51] Vanvuchelen N, Gijsbrechts J, Boute R. Use of proximal policy optimization for the joint replenishment problem. Comput Ind 2020;119:103239.
[52] Schulman J, Moritz P, Levine S, Jordan M, Abbeel P. High-dimensional continuous control using generalized advantage estimation. arXiv preprint 2015. arXiv:1506.02438.
[53] Kingma D, Ba J, Adam: a method for stochastic optimization. arXiv preprint 2014. arXiv:1412.6980.
[54] Andreeva-Mori A, Kubo D, Kobayashi K, Okuno Y, Homola J, Johnson M, et al. Operational testing of unmanned aircraft system traffic management in disaster response. J Air Transp 2021;29:160–70.